

{ - Linked lists Basics }

- { ① Explanation }
- { ② Pace → fast | slow → }
- { ③ }

Thursday - last session for IM.

Saturday -

PS { { PS Session at 9pm ✓

- Longest Pal. Substring
- Count Triplets

{ Ask any thing } → Ask anything that we discussed in class in the last 25 sessions.
(Conceptual) → Comparators

{ { } } → Send me before the { } }
cray on @slack

{ { Tuesday } - Advanced Module
{ { (6 Dec) }

{ { { { 23rd Dec - 3rd Jan } } } } { Break } }

- { ① Doubt Sessions - (You can ask previous session doubts) }
- { ② Time to time - chance to ask your doubts. }

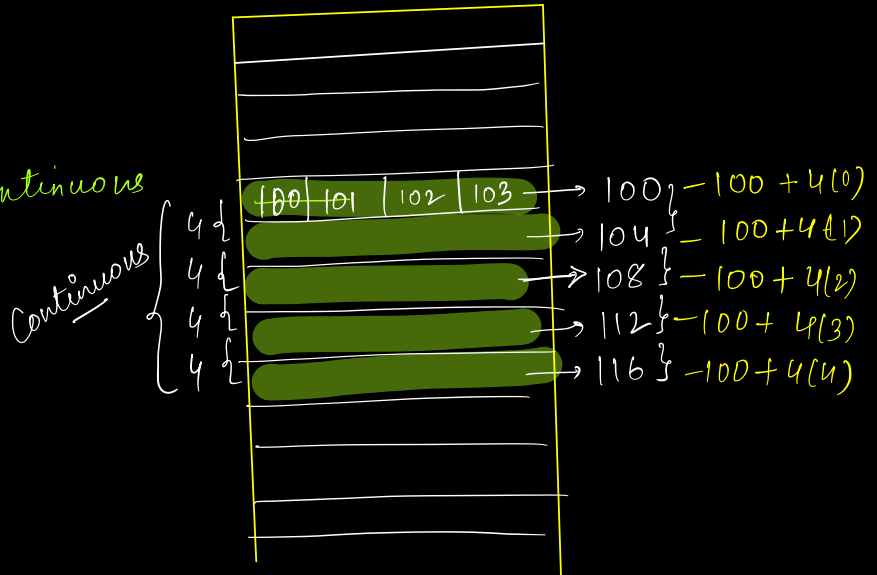
// Most Commonly Data Structure

- Arrays:

→ i^{th} index - $arr[i] = O(1)?$

int $arr[5] :-$
↓

it will allocate continuous memory blocks.



$$arr[i] = 100 + 4(i)$$

(Insertion in a list - $O(1)$) =
(avg)

array[↓]

list ✓ →

ls.add(1)
(2)
(3)
(4) ↓

→ arraylist.

← list - Dynamic Arrays

{arrays} ⇒

↳ int a[4] } 4 elements filled

↳ int a[8]

- copy a[4] → a[8]

- insert new element in a[8]

↓
O(N)

list Create

int a[N]

↓ All N positions filled

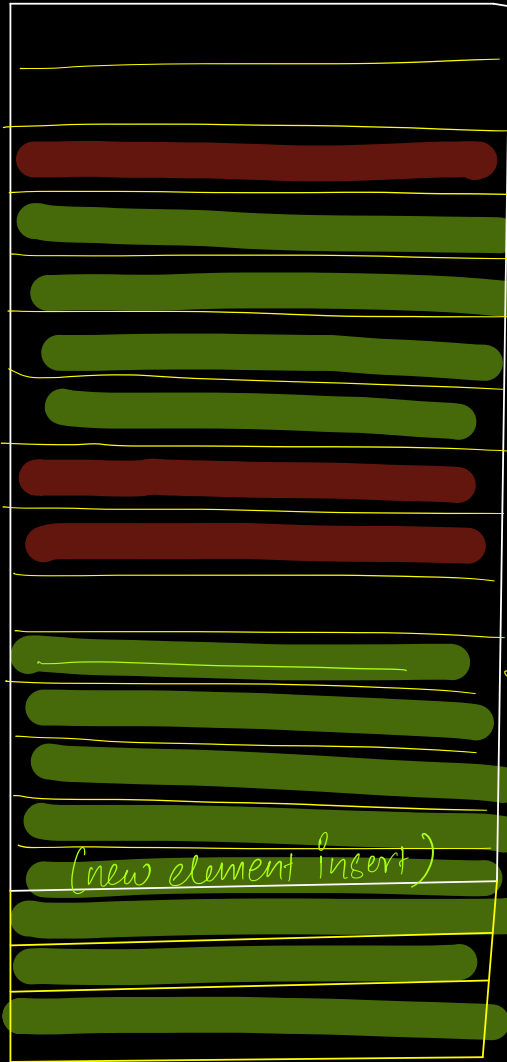
int a[2N]

copy all N elements from
a[] → a[]

↓
O(N)

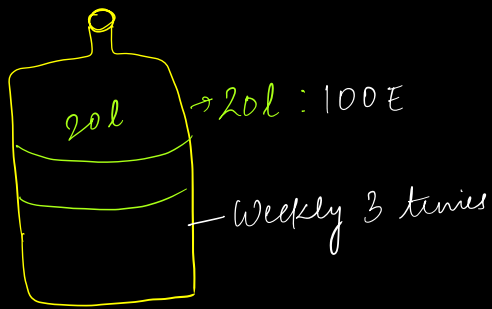
add new element in a[]

Unbuilt Implementation



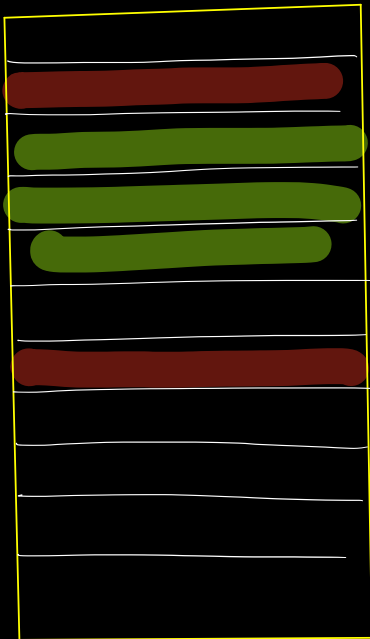
Worst case :- Insert in list - O(N)

5-10 years



One time Investment

Aquaguard :- 10,000



Dynamic array :-

`int ar[3]`

3 blocks



`int ar[6]` → 6 continuous blocks

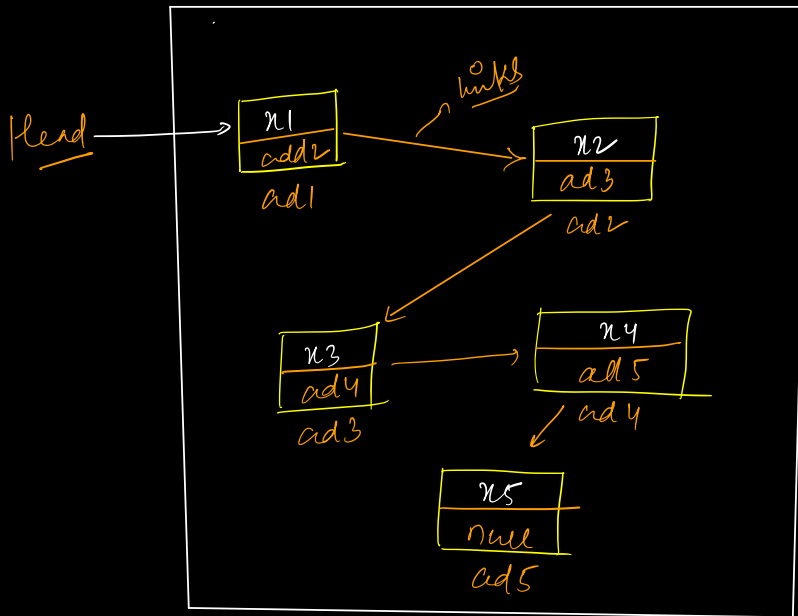
Will not be able to create

→ You want to properly memory

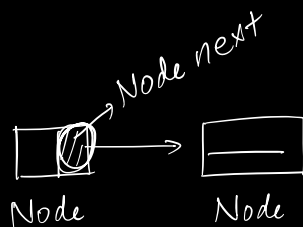
(You are ready to compromise $O(1)$ access time)

{ linked lists }

→ We can use random to -



Singly
Lk. }



→ User defined data type

```

class Node {
    int data;
    Node next; // We will store address of next node // Language
    Node (int n) { // Constructor
        data = n;
        next = NULL (keyword);
    }
}

```

Node temp = head;

print(temp)

Obj Ref

temp →

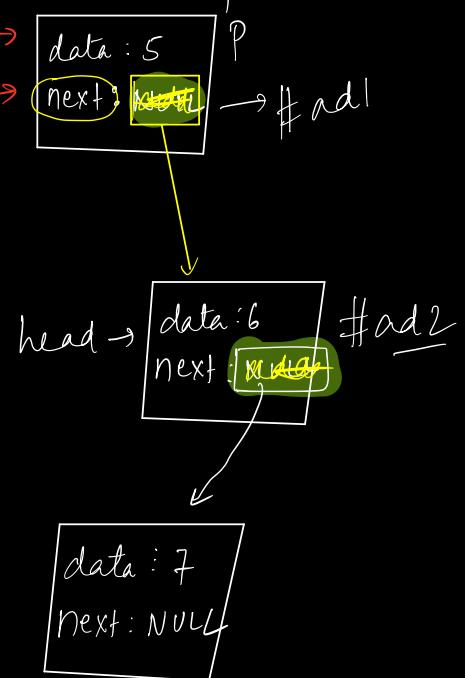
head →

```

main() {
    Node head = new Node(5);
    #ad1
    head.next = new Node(6);
    // head = head.next;
    head.next = new Node(7);
}

```

Obj Ref



{ Don't leave head node
address }

Node create linked list (int N) {

N=3: head → [1] → [2] → [3|NULL]

N=5: head → [1] → [2] → [3] → [4] → [5] → NULL

↓
{ Just return head node }

Node head = new Node(1); ← allocates memory & returns reference to it

Node temp = head

for(int i=2; i <= N; i++) {

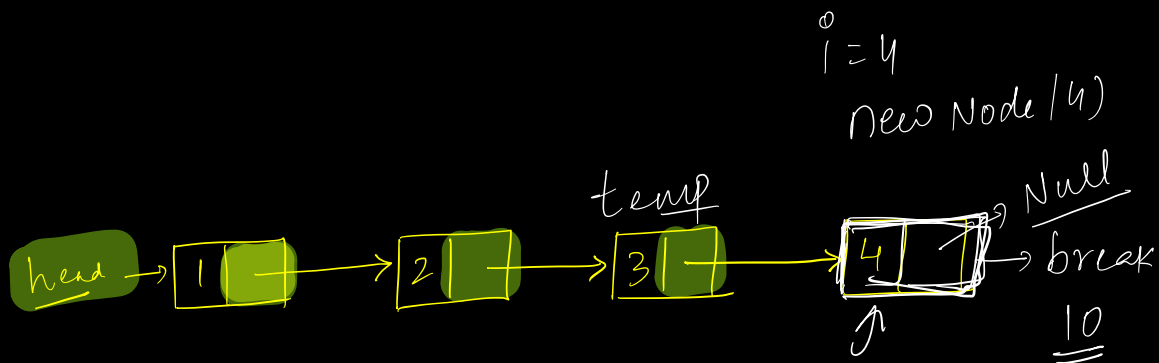
temp.next = new Node(i);

temp = temp.next;

}

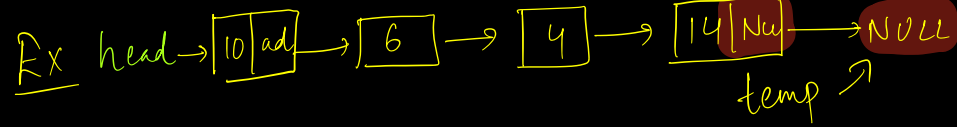
return head;

}



temp.next = new Node(10)
↓
break

```
int size (Node head) {
```



$\left. \begin{array}{l} \text{temp.data} \\ \text{temp.next} \end{array} \right\} \times$
↓
NULL

```
    c = 0
```

```
    Node temp = head; {good practice}
```

```
    int c = 0
```

```
    while (temp != NULL) {
```

```
        temp = temp.next;
```

```
        c++
```

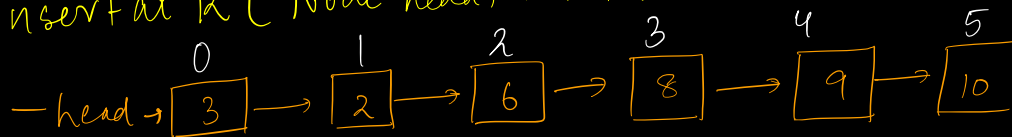
```
    }
```

```
    return c
```

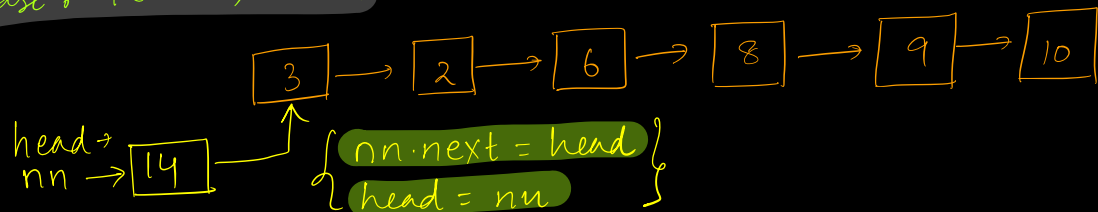
```
}
```

Insert at Node in a Lh:- (Consider 0 based)

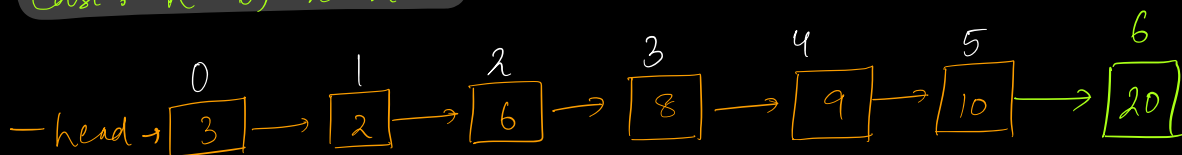
Insert at K (Node head, int K, int x)



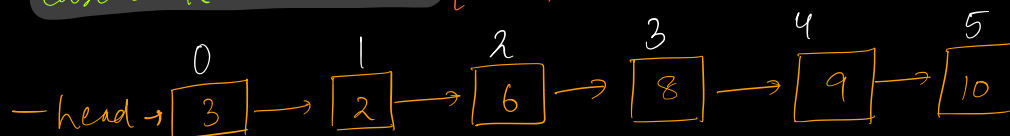
Case: $K=0, x=14$



Case: $K=6, x=20$



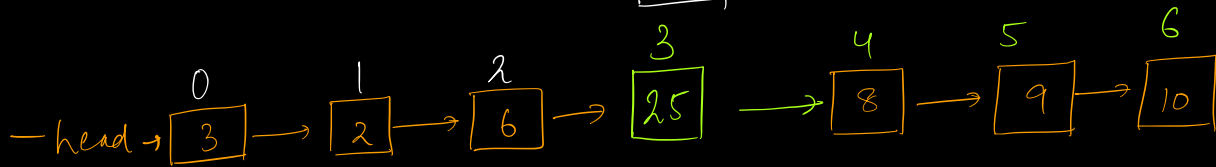
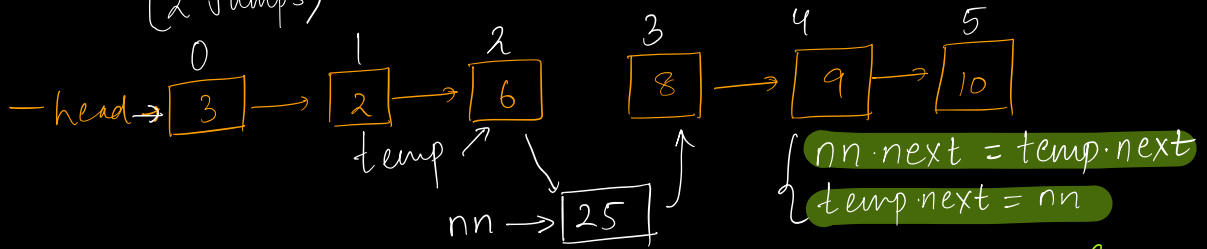
Case: $K=7, x=14$ {Not possible} if $(K > \text{len of Lh}) \rightarrow \text{cannot}$



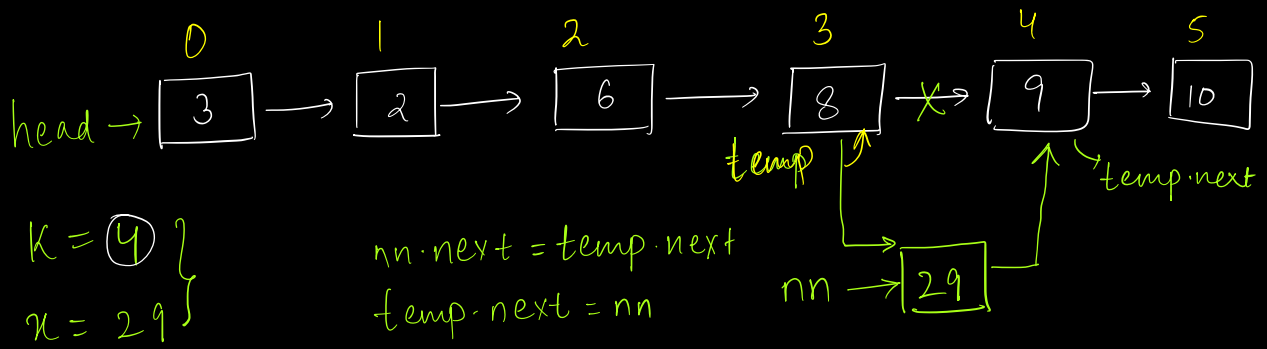
Case 4: $K=3, x=25$

Node nn = new Node(25)

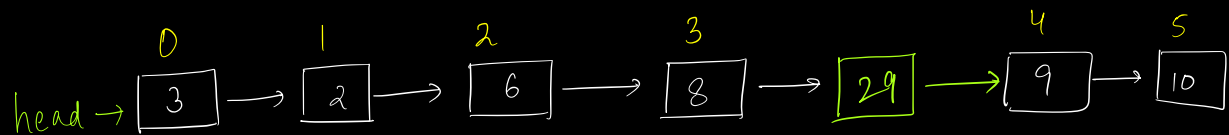
(2 Jumps)

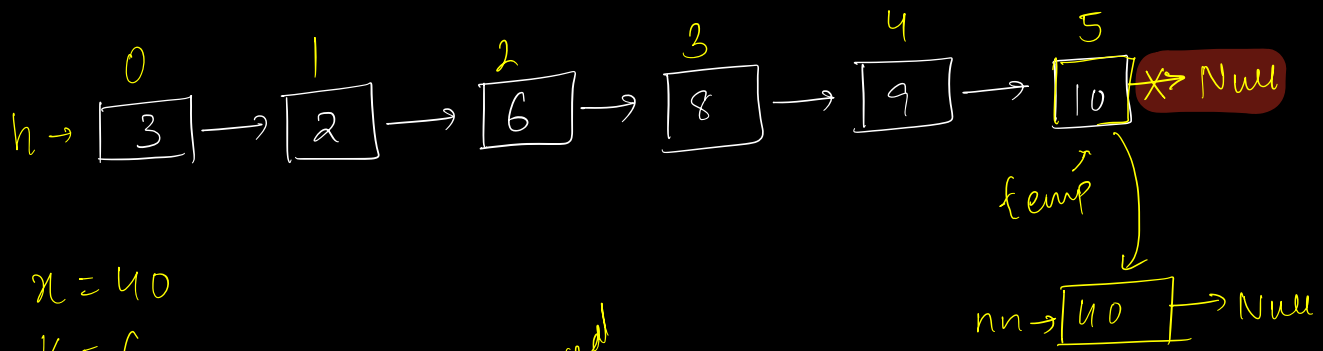


Kth pos



Node nn = new Node(29);





$x = 40$

$k = 6$

Already Covered

$\{ nn.next = temp.next ;$
 $\{ temp.next = nn ;$

Thurs - last Session (Tree Basics - 1) }
Sat - PS Session }

23rd Dec - 3rd Jan.

